



Drivers of Product Success and Revenue on Amazon

Presented by **Harika Veeramacheneni,**
Naisargi Doshi, Ruchi Patel, Srividya Peri

Database Marketing MKTG 638
March 4, 2026





Introduction

Database Description

Research Questions A - E

Recommendations

Appendix



Objective 01

Apply SQL to analyze real-world Amazon sales data.



Objective 02

Identify drivers of product revenue.



Objective 03

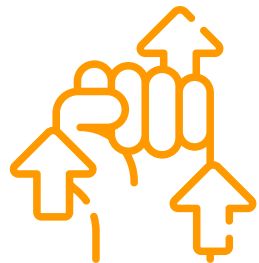
Evaluate impact of discounts, ratings, and pricing.



Objective 04

Provide strategic marketing insights.

Database Description



Number of Observations

$N \geq 200$



Key Variables

product_id,
product_category, price,
total_revenue,
customer_region,
payment_method,

discounted_price,
quantity_sold, rating,
review_count
order_id, order_date,
discount_percent



Source of Database

[kaggle.com](https://www.kaggle.com)

This dataset features Amazon e-commerce sales data that reflects realistic **time-series sales patterns**, pricing, discounts, customer regions, and ratings across multiple product categories.

Research Questions



What drives product success and revenue on Amazon?

A

Do high-quantity orders in North America represent the top-performing transactions in terms of units sold?

B

Do “Excellent” rated products generate significantly higher revenue than “Good” or “Poor” rated products?

C

What is the overall purchasing behavior across all Amazon orders, and is there a measurable relationship between `discount_percent` and `quantity_sold`?

D

Which product categories demonstrate strong revenue efficiency by generating an average revenue per order greater than 300?

E

Within each `customer_region`, how do `payment_methods` rank based on the number of orders, and which payment method is most preferred in each region?

RQ A: Do high-quantity orders in North America represent the top-performing transactions in terms of units sold?

	order_id bigint	order_date text	product_id bigint	product_category text	quantity_sold integer	total_revenue numeric (14,2)
1	27833	2022-03-04	3821	Sports	5	2499.55
2	18858	2022-06-20	1570	Sports	5	2487.30
3	20016	2022-02-02	3667	Electronics	5	2479.15
4	22400	2022-12-02	1024	Fashion	5	2472.30
5	36495	2022-12-23	1383	Beauty	5	2462.30
6	32159	2022-09-02	3994	Electronics	5	2454.50
7	44939	2023-04-09	3452	Books	5	2451.05
8	43794	2022-03-01	1580	Books	5	2434.70
9	35373	2022-03-07	1404	Books	5	2428.05
10	1077	2023-06-01	2074	Fashion	5	2419.35

```
SELECT order_id, product_id, product_category,  
quantity_sold, total_revenue  
FROM sales  
WHERE customer_region = 'North America'  
ORDER BY quantity_sold DESC  
  
LIMIT 10
```

KEY FINDINGS:

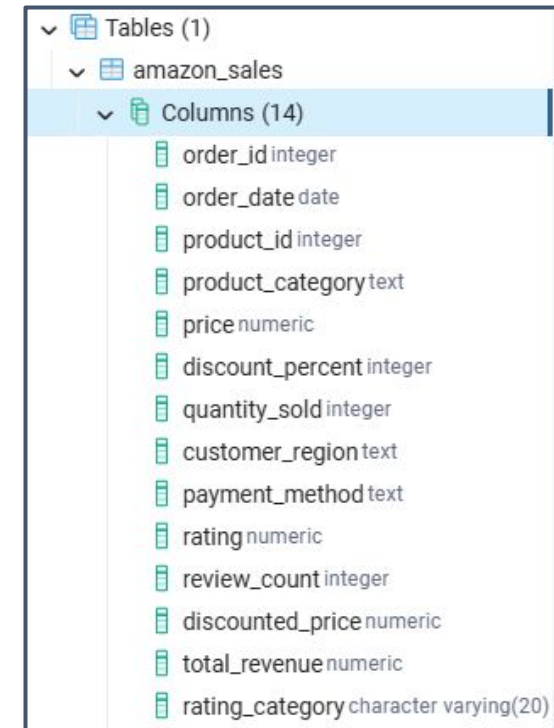
- All top 10 North American transactions recorded the highest quantity_sold value of 5 units, confirming they represent the strongest unit-level performance.
- The total revenue from these transactions was approximately \$24,588, with the top product contributing 10.17% of total top-10 revenue.
- Revenue is distributed across multiple categories (Sports, Electronics, Fashion, Books, Beauty), indicating diversified product performance rather than reliance on a single dominant item.

RQ B: Do “Excellent” rated products generate significantly higher revenue than “Good” or “Poor” rated products?

```
UPDATE amazon_sales
SET rating_category =
CASE
  WHEN rating >= 4.5 THEN 'Excellent'
  WHEN rating BETWEEN 3.5 AND 4.49 THEN 'Good'
  ELSE 'Poor'
END;
```

KEY FINDINGS:

- Revenue differences are driven by order volume, not spending behaviour.
- The difference in average revenue is less than 1% (about \$2–6 per order).
- “Excellent” products generate slightly higher revenue per order, data suggests a weak relationship between rating and revenue.



Tables (1)

amazon_sales

Columns (14)

- order_id integer
- order_date date
- product_id integer
- product_category text
- price numeric
- discount_percent integer
- quantity_sold integer
- customer_region text
- payment_method text
- rating numeric
- review_count integer
- discounted_price numeric
- total_revenue numeric
- rating_category character varying(20)

	rating_category character varying (20) 🔒	total_orders bigint 🔒	avg_revenue numeric 🔒	total_revenue numeric 🔒
1	Excellent	6798	661.40	4496210.88
2	Good	12622	659.58	8325204.52
3	Poor	30580	655.50	20045158.34

RQ C: What is the overall purchasing behavior across all Amazon orders, and is there a measurable relationship between discount_percent and quantity_sold?

	total_orders bigint	min_quantity integer	max_quantity integer	avg_quantity numeric	quantity_std_dev numeric	discount_quantity_correlation double precision
1	50000	1	5	2.9994000000000000	1.4154008998139516	0.0013989282759390089

SELECT

```
COUNT(quantity_sold) AS total_orders,  
MIN(quantity_sold) AS min_quantity,  
MAX(quantity_sold) AS max_quantity,  
AVG(quantity_sold) AS avg_quantity,  
STDDEV(quantity_sold) AS quantity_std_dev,  
CORR(discount_percent, quantity_sold) AS  
discount_quantity_correlation  
FROM amazon_sales;
```

KEY FINDINGS:

- Purchasing behavior is consistent and predictable across orders.
- No strong evidence of bulk buying patterns.
- Discounts do not significantly influence purchase quantities.
- Buying decisions appear need-driven rather than discount-driven.

RQ D: Which product categories demonstrate strong revenue efficiency by generating an average revenue per order greater than 300?

	product_category text	avg_revenue_per_order numeric
1	Home & Kitchen	662.7673225962702834
2	Books	658.6841635643088747
3	Electronics	657.5233209134615385
4	Beauty	655.7147040756054341
5	Fashion	655.1253245666467424
6	Sports	654.2330090744101633

```
SELECT
    product_category,
    AVG(total_revenue) AS avg_revenue_per_order
FROM amazon_salesfinal
GROUP BY product_category
HAVING AVG(total_revenue) > 300
ORDER BY avg_revenue_per_order
DESC;
```

KEY FINDINGS:

- All categories exceed \$300 average revenue per order (approx \$654–\$663).
- Home & Kitchen leads slightly; overall performance is very similar across categories.
- Revenue is balanced and diversified across the product portfolio.

RQ E: Within each customer_region, how do payment_methods rank based on the number of orders, and which payment method is most preferred in each region?

```
SELECT
    customer_region,
    payment_method,
    order_count,
    RANK() OVER (
        PARTITION BY customer_region
        ORDER BY order_count DESC
    ) AS payment_rank
FROM (
    SELECT
        customer_region,
        payment_method,
        COUNT(order_id) AS order_count
    FROM amazon_sales
    GROUP BY customer_region,
    payment_method
) sub;
```

	customer_region text	payment_method text	order_count bigint	payment_rank bigint
1	Asia	UPI	2606	1
2	Asia	Wallet	2496	2
3	Asia	Cash on Delivery	2487	3
4	Asia	Credit Card	2486	4
5	Asia	Debit Card	2451	5
6	Europe	Debit Card	2551	1
7	Europe	Wallet	2544	2
8	Europe	UPI	2478	3
9	Europe	Credit Card	2468	4
10	Europe	Cash on Delivery	2411	5
11	Middle East	Cash on Delivery	2544	1
12	Middle East	Wallet	2526	2
13	Middle East	Credit Card	2493	3
14	Middle East	Debit Card	2486	4
15	Middle East	UPI	2456	5
16	North America	Wallet	2540	1
17	North America	UPI	2538	2
18	North America	Debit Card	2493	3
19	North America	Cash on Delivery	2485	4
20	North America	Credit Card	2461	5

KEY FINDINGS:

- North America shows strong digital wallet adoption.
- Asia demonstrates preference for instant digital payments (UPI).
- Europe leans toward traditional debit-based transactions.
- Middle East maintains strong reliance on Cash on Delivery.

Recommendations

01

Diversified Product Promotion

Highlight multiple high-performing products across categories rather than relying on a single flagship item to drive sales to boost broader purchasing behaviour.

02

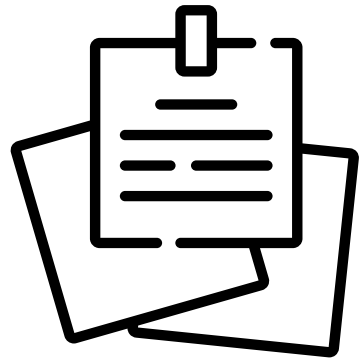
Performance-based Segmentation

Combine rating with stronger performance indicators to make impactful marketing decisions.

03

Promotional Strategy

Instead of broad discounting strategies, Amazon should focus on targeted, demand-driven promotions tailored to specific products or regions.



04

Cross-targeting

Maintain the balanced, high-performing product mix while focusing on upselling, bundling, and targeted cross-selling to increase average order value, with slight additional support for top categories like Home & Kitchen.

05

Payment Method Strategy

Optimize checkout strategy and promotional campaigns regionally by prioritizing the top-ranked payment method in each market.

amazon

THANK YOU!



Appendix RQ A

```
21 SELECT
22     order_id,
23     order_date,
24     product_id,
25     product_category,
26     quantity_sold,
27     total_revenue
28 FROM sales
29 WHERE customer_region = 'North America'
30 ORDER BY quantity_sold DESC, total_revenue DESC
31 LIMIT 10;
```

```
32 WITH top10_na AS (
33     SELECT
34         order_id,
35         product_id,
36         product_category,
37         quantity_sold,
38         total_revenue
39     FROM sales
40     WHERE customer_region = 'North America'
41     ORDER BY quantity_sold DESC, total_revenue DESC
42     LIMIT 10
43 )
44 SELECT
45     product_id,
46     product_category,
47     SUM(quantity_sold) AS units_sold_in_top10,
48     SUM(total_revenue) AS revenue_in_top10,
49     COUNT(*) AS rows_in_top10
50 FROM top10_na
51 GROUP BY product_id, product_category
52 ORDER BY units_sold_in_top10 DESC, revenue_in_top10 DESC;
```

```
53 WITH top10_na AS (
54     SELECT product_id, total_revenue
55     FROM sales
56     WHERE customer_region = 'North America'
57     ORDER BY quantity_sold DESC, total_revenue DESC
58     LIMIT 10
59 ),
60 product_rev AS (
61     SELECT product_id, SUM(total_revenue) AS product_revenue
62     FROM top10_na
63     GROUP BY product_id
64 ),
65 totals AS (
66     SELECT SUM(total_revenue) AS top10_total_revenue
67     FROM top10_na
68 )
69 SELECT
70     pr.product_id,
71     pr.product_revenue,
72     t.top10_total_revenue,
73     ROUND(100.0 * pr.product_revenue / t.top10_total_revenue, 2) AS pct_of_top10_revenue
74 FROM product_rev pr
75 CROSS JOIN totals t
76 ORDER BY pr.product_revenue DESC
77 LIMIT 1;
```

Step 1: Filter for North America transactions.

- Used a **WHERE** clause to isolate transactions where **customer_region** = 'North America'.
- This ensures the analysis focuses only on the target geographic market.

Step 2: Rank transactions by units sold.

- Used **ORDER BY quantity_sold DESC** to sort transactions from highest to lowest unit volume.
- This identifies which transactions represent the strongest performance in terms of units sold.

Step 3: Limit results to top-performing transactions.

- Applied **LIMIT 10** to retrieve only the top 10 highest-quantity transactions.
- This allows direct evaluation of the strongest-performing orders within North America.

Step 4: Evaluate product and revenue concentration.

- Grouped results by **product_id** and **product_category**.
- **SUM(quantity_sold)** identifies best-selling products within the top 10.
- **SUM(total_revenue)** measures total revenue contribution per product.
- Revenue share calculation determines whether performance is concentrated in a single product or distributed across multiple categories.

Appendix RQ B

```
Query  Query History
1  ALTER TABLE amazon_sales
2  ADD rating_category VARCHAR(20);
3
4  UPDATE amazon_sales
5  SET rating_category =
6  CASE
7      WHEN rating >= 4.5 THEN 'Excellent'
8      WHEN rating BETWEEN 3.5 AND 4.49 THEN 'Good'
9      ELSE 'Poor'
10  END;
11
12 SELECT
13     rating_category,
14     COUNT(*) AS total_orders,
15     ROUND(AVG(total_revenue), 2) AS avg_revenue,
16     ROUND(SUM(total_revenue), 2) AS total_revenue
17 FROM amazon_sales
18 GROUP BY rating_category
19 ORDER BY avg_revenue DESC;
20
```

Tables (1)	
amazon_sales	
Columns (14)	
order_id	integer
order_date	date
product_id	integer
product_category	text
price	numeric
discount_percent	integer
quantity_sold	integer
customer_region	text
payment_method	text
rating	numeric
review_count	integer
discounted_price	numeric
total_revenue	numeric
rating_category	character varying(20)

Step 1: Create a new column titled “rating_category”.

- Added a new column, **rating_category**.
- Column allows numerical ratings to be compared to categorical labels.

Step 2: UPDATE with CASE.

- Assigned each product a category based on rating.
- Created comparable groups which were easy to interpret.

Step 3: SELECT Query.

- **Rating_category**: categories being compared.
- **COUNT(*) AS total_orders**: number of orders in each category.
- **AVG(total_revenue)**: averages revenue for each order, for each category.
- **SUM(total_revenue)**: shows total revenue for each category.

Appendix RQ C

Step 1: Analyze overall purchasing behavior.

Applied aggregate functions on the `quantity_sold` column to evaluate order patterns across all transactions.

- `COUNT` determined total orders analyzed.
- `MIN` and `MAX` identified the smallest and largest order sizes.
- `AVG` measured the typical units purchased per order.
- `STDDEV` assessed variability in purchasing behavior.

Step 2: Assess discount impact

- Used `CORR(discount_percent, quantity_sold)` to measure the linear relationship between discounts and order volume.
- A correlation value close to:
 - +1 indicates strong positive relationship
 - -1 indicates strong negative relationship
 - 0 indicates no meaningful linear relationship

Step 3: Interpret statistical consistency.

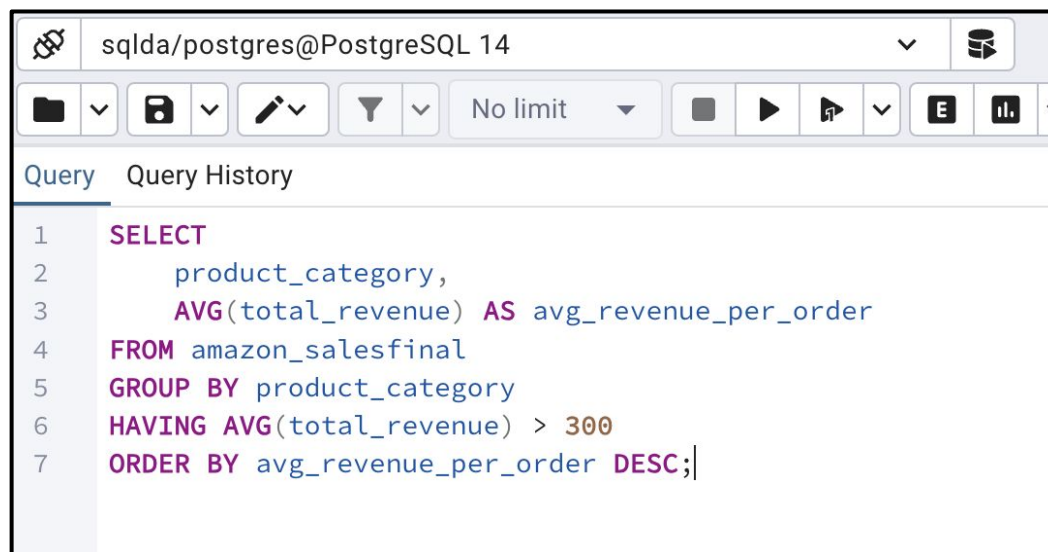
- Compared the average quantity with the standard deviation to assess whether purchasing behavior is stable or highly variable.
- This helps determine whether bulk purchasing patterns are widespread or limited.

Step 4: Assess strategic implications.

- Based on the correlation result, evaluated whether discount strategies are effective in driving higher purchase volumes.
- This step connects statistical findings to business decision-making.

```
SELECT
    COUNT(quantity_sold) AS total_orders,
    MIN(quantity_sold) AS min_quantity,
    MAX(quantity_sold) AS max_quantity,
    AVG(quantity_sold) AS avg_quantity,
    STDDEV(quantity_sold) AS quantity_std_dev,
    CORR(discount_percent, quantity_sold) AS discount_quantity_correlation
FROM amazon_sales;
```

Appendix RQ D



The screenshot shows a SQL client window titled 'sqlda/postgres@PostgreSQL 14'. The interface includes a toolbar with icons for file operations, a filter icon, and a 'No limit' dropdown. Below the toolbar, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL query with line numbers 1 through 7. The query is as follows:

```
1 SELECT
2     product_category,
3     AVG(total_revenue) AS avg_revenue_per_order
4 FROM amazon_salesfinal
5 GROUP BY product_category
6 HAVING AVG(total_revenue) > 300
7 ORDER BY avg_revenue_per_order DESC;
```

Step 1: Aggregate revenue by product category

- Used **AVG(total_revenue)** to calculate average revenue per order.
- Grouped results using **GROUP BY product_category**.

Step 2: Apply performance threshold

- Used **HAVING AVG(total_revenue) > 300**.
- Filtered categories to identify strong revenue efficiency.

Step 3: Rank results

- Applied **ORDER BY avg_revenue_per_order DESC**.
- Ranked categories from highest to lowest average revenue.

Appendix RQ E

Step 1: Aggregate order counts by region and payment method.

- Used a subquery to group transactions by **customer_region** and **payment_method**.
- Applied **COUNT(order_id)** to calculate total orders for each payment method within each region.

This provides the transaction volume required to evaluate payment preferences.

Step 2: Rank payment methods within each region.

Applied the **RANK()** window function with:

- **PARTITION BY customer_region** to reset ranking for each region, and
- **ORDER BY order_count DESC** to rank payment methods from highest to lowest order volume.
- This identifies the most preferred payment method in each region.

Step 3: Compare regional payment patterns.

- Analyzed ranking results to determine which payment method holds the top position in each region and whether preferences are concentrated or evenly distributed.

Step 4: Derive strategic implications.

- Used the ranking outcomes to assess regional payment behavior and identify opportunities for localized payment optimization strategies.

```
SELECT
    customer_region,
    payment_method,
    order_count,
    RANK() OVER (
        PARTITION BY customer_region
        ORDER BY order_count DESC
    ) AS payment_rank
FROM (
    SELECT
        customer_region,
        payment_method,
        COUNT(order_id) AS order_count
    FROM amazon_sales
    GROUP BY customer_region, payment_method
) sub;
```